

inverse (or inverse of the transpose) of $\mathbf{M}$,

$$(\mathbf{M}^{-1})^T = \begin{bmatrix} 0 & 0 & \dots & 0 & 0 & 1 \\ 1 & 0 & \dots & 0 & 0 & c_1 \\ 0 & 1 & \dots & 0 & 0 & c_2 \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 & c_{n-2} \\ 0 & 0 & \dots & 0 & 1 & c_{n-1} \end{bmatrix} \quad (7.5.11)$$

Surprise! This is exactly Method II, as also shown in Figure 7.5.1(b). (Work it out.)

Even more specifically, the sequence of bits output by a Method II LFSR based on a primitive polynomial $P(x)$ is identical to the sequence output by a Method I LFSR that uses the *reciprocal polynomial* $x^n P(1/x)$. The proof is a bit beyond our scope, but it is essentially because the matrix $\mathbf{M}$ and its transpose are both roots of the characteristic polynomial, equation (7.5.2), while the inverse matrix $\mathbf{M}^{-1}$ and its transpose are both roots of the reciprocal polynomial. The reciprocal polynomial, as you can easily check from the definition, just swaps the positions of nonzero coefficients end-to-end. For example, the reciprocal polynomial of equation (7.5.3) is (18, 17, 16, 13, 1). If a polynomial is primitive, so is its reciprocal.

Try this experiment: Run a Method II generator for a while. Then take n consecutive bits of its output (from its highest bit, say) and put them into a Method I shift register as initialization (low bit the most recent one). Now step the two methods together, using the reciprocal polynomial in the Method I. You'll get identical output from the two generators.

CITED REFERENCES AND FURTHER READING:

- Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), pp. 30ff.
- Horowitz, P., and Hill, W. 1989, *The Art of Electronics*, 2nd ed. (Cambridge, UK: Cambridge University Press), §9.32 – §9.37.
- Tausworthe, R.C. 1965, "Random Numbers Generated by Linear Recurrence Modulo Two," *Mathematics of Computation*, vol. 19, pp. 201–209.
- Watson, E.J. 1962, "Primitive Polynomials (Mod 2)," *Mathematics of Computation*, vol. 16, pp. 368–369.[1]

7.6 Hash Tables and Hash Memories

It's a strange dream. You're in a kind of mailroom whose walls are lined with numbered pigeonhole boxes. A man, Mr. Hacher, sits at a table. You are standing. There is an in-basket mounted on the wall. Your job is to take letters from the in-basket and sort them into the pigeonholes.

But how? The letters are addressed by name, while the pigeonholes are only numbered. That is where Mr. Hacher comes in. You show him each letter, and he immediately tells you its pigeonhole number. He always gives the same number for the same name, while different names always get different numbers (and therefore unique pigeonholes).

Over time, as the number of addressees grows, there are fewer and fewer empty boxes until, finally, none at all. This is not a problem as long as letters arrive only for existing boxholders. But one day, you spot a new name on an envelope. With trepidation you put it in front of Mr. Hacher . . . and you wake up!

Mr. Hacher and his table are a *hash table*. A hash table behaves as if it keeps a running ledger of all the *hash keys* (the addressee names) that it has ever seen, assigns a unique number to each, and is able to look through all the names for every new query, either returning the same number as before (for a repeat key) or, for a new key, assigning a new one. There is usually also an option to erase a key.

The goal in implementing a hash table is to make all these functions take only a few computer operations each, not even $O(\log N)$. That is quite a trick, if you think about it. Even if you somehow maintain an ordered or alphabetized list of keys, it will still take $O(\log N)$ operations to find a place in the list, by bisection, say. The big idea behind hash tables is the use of random number techniques (§7.1) to map a hash key to a pseudo-random integer between 0 and $N - 1$, where N is the total number of pigeonholes. Here we definitely want pseudo-random and not random integers, because the same key must produce the same integer each time.

In first approximation, ideally much of the time, that initial pseudo-random integer, called the output of the *hash function*, or (for short) the key's *hash*, is what the hash table puts out, i.e., the number given out by Mr. Hacher. However, it is possible that, by chance, two keys have the same hash; in fact this becomes increasingly probable as the number of distinct keys approaches N , and a certainty when N is exceeded (the *pigeonhole principle*). The implementation of a hash table therefore requires a *collision strategy* that ensures that unique integers are returned, even for (different) keys that have the same hash.

Many vendors' implementations of the C++'s Standard Template Library (STL) provide a hash table as the class `hash_map`. Unfortunately, at this writing, `hash_map` is not a part of the actual STL standard, and the quality of vendor implementations is also quite variable. We therefore here implement our own; thereby we can both learn more about the principles involved and build in some specific features that will be useful later in this book (for example §21.8 and §21.6).

7.6.1 Hash Function Object

By a hash function object we mean a structure that combines a hashing algorithm (as in §7.1) with the “glue” needed to make a hash table. The object should map an arbitrary key type `keyT`, which itself may be a structure containing multiple data values, into (for our implementation) a pseudo-random 64-bit integer. All the hash function object really needs to know about `keyT` is its length in bytes, that is, `sizeof(keyT)`, since it doesn't care how those bytes are used, only that they are part of the key to be hashed. We therefore give the hash function object a constructor that tells it how many bytes to hash; and we let it access a key by a void pointer to the key's address. Thus the object can access those bytes any way it wants.

As a first example of a hash function object, let's just put a wrapper around the hash function algorithm of §7.1.4. This is quite efficient when `sizeof(keyT) = 4` or 8.

```
struct Hashfn1 {
```

Example of an object encapsulating a hash function for use by the class `Hashmap`.

```
    Ranhash hasher;
```

The actual hash function.

```
    Int n;
```

Size of key in bytes.

```
    Hashfn1(Int nn) : n(nn) {}
```

Constructor just saves key size.

```
    Ullong fn(const void *key) {
```

Function that returns hash from key.

```
        Uint *k;
```

```
        Ullong *kk;
```

`hash.h`

```

switch (n) {
    case 4:
        k = (UInt *)key;
        return hasher.int64(*k);           Return 64-bit hash of 32-bit key.
    case 8:
        kk = (Ullong *)key;
        return hasher.int64(*kk);         Return 64-bit hash of 64-bit key.
    default:
        throw("Hashfn1 is for 4 or 8 byte keys only.");
}
};

```

(Since n is constant for the life of the object, it's a bit inefficient to be testing it on every call; you should edit out the unnecessary code when you know n in advance.)

More generally, a hash function object can be designed to work on arbitrary sized keys by incorporating them into a final hash value a byte at a time. There is a trade-off between speed and degree-of-randomness. Historically, hash functions have favored speed, with simple incorporation rules like

$$\begin{aligned}
 h_0 &= \text{some fixed constant} \\
 h_i &= (m h_{i-1} \text{ op } k_i) \bmod 2^{32} \quad (i = 1 \dots K)
 \end{aligned}
 \tag{7.6.1}$$

Here k_i is the i th byte of the key ($1 \leq i \leq K$), m is a multiplier with popular values that include 33, 63689, and $2^{16} + 2^6 - 1$ (doing the multiplication by shifts and adds in the first and third cases), and “op” is either addition or bitwise XOR. You get the mod function for free when you use 32-bit unsigned integer arithmetic. However, since 64-bit arithmetic is fast on modern machines, we think that the days of small multipliers, or many operations changing only a few bits at a time, are over. We favor hash functions that can pass good tests for randomness. (When you know a lot about your keys, it is possible to design hash functions that are even *better* than random, but that is beyond our scope here.)

A hash function object may also do some initialization (of tables, etc.) when it is created. Unlike a random number generator, however, it may not store any history-dependent state between calls, because it must return the same hash for the same key every time. Here is an example of a self-contained hash function object for keys of any length. This is the hash function object that we will use below.

```

hash.h  struct Hashfn2 {
        Another example of an object encapsulating a hash function, allowing arbitrary fixed key sizes
        or variable-length null terminated strings. The hash function algorithm is self-contained.
        static Ullong hashfn_tab[256];
        Ullong h;
        Int n;
        Hashfn2(Int nn) : n(nn) {
            if (n == 1) n = 0;
            h = 0x544B2FBACAAAF1684LL;
            for (Int j=0; j<256; j++) {
                for (Int i=0; i<31; i++) {
                    h = (h >> 7) ^ h;
                    h = (h << 11) ^ h;
                    h = (h >> 10) ^ h;
                }
                hashfn_tab[j] = h;
            }
        }
};

```

Size of key in bytes, when fixed size.

Null terminated string key signaled by $n = 0$ or 1.

Length 256 lookup table is initialized with values from a 64-bit Marsaglia generator stepped 31 times between each.

```

Ullong fn(const void *key) {           Function that returns hash from key.
    Int j;
    char *k = (char *)key;           Cast the key pointer to char pointer.
    h=0xBB40E64DA205B064LL;
    j=0;
    while (n ? j++ < n : *k) {         Fixed length or else until null.
        h = (h * 7664345821815920749LL) ^ hashfn_tab[(unsigned char)(*k)];
        k++;
    }
    return h;
}
};
Ullong Hashfn2::hashfn_tab[256];      Defines storage for the lookup table.

```

The method used is basically equation (7.6.1), but (i) with a large constant that is known to be a good multiplier for a linear congruential random number generator mod 2^{64} , and, more importantly, (ii) a table lookup that substitutes a random (but fixed) 64-bit value for every byte value in $0 \dots 255$. Note also the tweak that allows Hashfn2 to be used either for fixed length key types (call constructor with $n > 1$) or with null terminated byte arrays of variable length (call constructor with $n = 0$ or 1).

7.6.2 Hash Table

By *hash table* we mean an object with the functionality of Mr. Hacher (and his table) in the dream, namely to turn arbitrary keys into unique integers in a specified range. Let's dive right in. In outline, the Hashtable object is

```

template<class keyT, class hfnT> struct Hashtable {           hash.h
    Instantiate a hash table, with methods for maintaining a one-to-one correspondence between
    arbitrary keys and unique integers in a specified range.
    Int nhash, nmax, nn, ng;
    VecInt htable, next, garbg;
    VecUllong thehash;
    hfnT hash;                                                 An instance of a hash function object.
    Hashtable(Int nh, Int nv);
    Constructor. Arguments are size of hash table and max number of stored elements (keys).

    Int iget(const keyT &key);                                Return integer for a previously set key.
    Int iset(const keyT &key);                                Return unique integer for a new key.
    Int ierase(const keyT &key);                               Erase a key.
    Int ireserve();                                           Reserve an integer (with no key).
    Int irelinquish(Int k);                                    Un-reserve an integer.
};

template<class keyT, class hfnT>
Hashtable<keyT,hfnT>::Hashtable(Int nh, Int nv):
    Constructor. Set nhash, the size of the hash table, and nmax, the maximum number of elements
    (keys) that can be accommodated. Allocate arrays appropriately.
    hash(sizeof(keyT)), nhash(nh), nmax(nv), nn(0), ng(0),
    htable(nh), next(nv), garbg(nv), thehash(nv) {
        for (Int j=0; j<nhash; j++) { htable[j] = -1; }      Signifies empty.
    }

```

A Hashtable object is templated by two class names: the class of the key (which may be as simple as `int` or as complicated as a multiply derived class) and the class of the hash function object (e.g., `Hashfn1` or `Hashfn2`, above). Note how the hash function object is automatically created using the size of `keyT`, so the user is not responsible for knowing this value. If you are going to use variable length, null

terminated byte arrays as keys, then the type of `keyT` is `char`, not `char*`; see §7.6.5 for an example.

The hash table object is created from two integer parameters. The most important one is `nm`, the maximum number of objects that can be stored — in the dream, the number of pigeonholes in the room. For now, suppose that the second parameter, `nh`, has the same value as `nm`.

The overall scheme is to convert arbitrary keys into integers in the range $0 \dots nh-1$ that index into the array `htable`, by taking the output of the hash function modulo `nh`. That array's indexed element contains either -1 , meaning “empty,” or else an index in the range $0 \dots nm-1$ that points into the arrays `thehash` and `next`. (For a computer science flavor one could do this with list elements linked by pointers, but in the spirit of numerical computation, we will use arrays; both ways are about equally efficient.)

An element in `thehash` contains the 64-bit hash of whatever key was previously assigned to that index. We will take the identity of two hashes as being positive proof that their keys were identical. Of course this is not really true. There is a probability of $2^{-64} \sim 5 \times 10^{-20}$ of two keys giving identical hashes by chance. To guarantee error-free performance, a hash table must in fact store the actual key, not just the hash; but for our purposes we will accept the very small chance that two elements might get confused. (Don't use these routines if you are typically storing more than a billion elements in a single hash table. But you already knew that!)

This 10^{-20} coincidence is *not* what is meant by *hash collision*. Rather, hash collisions occur when two hashes yield the same value modulo `nh`, so that they point to the same element in `htable`. That is not at all unusual, and we must provide for handling it. Elements in the array `next` contain values that index back into `thehash` and `next`, i.e., form a linked list. So, when two or more keys have landed on the same value i , $0 \leq i < nh$, and we want to retrieve a particular one of them, it will either be in the location `thehash[i]`, or else in the (hopefully short) list that starts there and is linked by `next[i]`, `next[next[i]]`, and so forth.

We can now say more about the value that should be initially specified for the parameter `nh`. For a full table with all `nm` values assigned, the linked lists attached to each element of `htable` have lengths that are Poisson distributed with a mean $\lambda \equiv nm/nh$. Thus, large λ (`nh` too small) implies a lot of list traversal, while small λ (`nh` too large) implies wasted space in `htable`. Conventional wisdom is to choose $\lambda \sim 0.75$, in which case (assuming a good hash function) 47% of `htable` will be empty, 67% of the nonempty elements will have lists of length one (i.e., you get the correct key on the first try), and the mean number of indirections (steps in traversing the `next` pointers) is 0.42. For $\lambda = 1$, that is, `nh = nm`, the values are 37% table empty, 58% first try hits, and 0.58 mean indirections. So, in this general range, any choice is basically fine. The general formulas are

$$\begin{aligned} \text{empty fraction} &= P_\lambda(0) = e^{-\lambda} \\ \text{first try hits} &= P_\lambda(1)/[1 - P_\lambda(0)] = \frac{\lambda e^{-\lambda}}{1 - e^{-\lambda}} \\ \text{mean indirections} &= \sum_{j=2}^{\infty} \frac{(j-1)P_\lambda(j)}{1 - P_\lambda(0)} = \frac{e^{-\lambda} - 1 + \lambda}{1 - e^{-\lambda}} \end{aligned} \quad (7.6.2)$$

where $P_\lambda(j)$ is the Poisson probability function.

Now to the implementations within `Hashtable`. The simplest to understand is the “get” function, which returns an index value only if the key was previously “set,” and returns `-1` (by convention) if it was not. Our data structure is designed to make this as fast as possible.

```
template<class keyT, class hfnT>                                     hash.h
Int Hashtable<keyT,hfnT>::iget(const keyT &key) {
Returns integer in 0..nmax-1 corresponding to key, or -1 if no such key was previously stored.
    Int j,k;
    Ullong pp = hash.fn(&key);                                     Get 64-bit hash
    j = (Int)(pp % nhash);                                         and map it into the hash table.
    for (k = htable[j]; k != -1; k = next[k]) {                   Traverse linked list until an ex-
        if (theshash[k] == pp) {                                    act match is found.
            return k;
        }
    }
    return -1;                                                     Key was not previously stored.
}
```

A language subtlety to be noted is that `iget` receives `key` as a `const` reference, and then passes its *address*, namely `&key`, to the hash function object. C++ allows this, because the hash function object’s void pointer argument is itself declared as `const`.

The routine that “sets” a key is slightly more complicated. If the key has previously been set, we want to return the same value as the first time. If it hasn’t been set, we initialize the necessary links for the future.

```
template<class keyT, class hfnT>                                     hash.h
Int Hashtable<keyT,hfnT>::iset(const keyT &key) {
Returns integer in 0..nmax-1 that will henceforth correspond to key. If key was previously set,
return the same integer as before.
    Int j,k,kprev;
    Ullong pp = hash.fn(&key);                                     Get 64-bit hash
    j = (Int)(pp % nhash);                                         and map it into the hash table.
    if (htable[j] == -1) {                                         Key not in table. Find a free integer, either
        k = ng ? garbg[--ng] : nn++ ;                               new or previously erased.
        htable[j] = k;
    } else {                                                       Key might be in table. Traverse list.
        for (k = htable[j]; k != -1; k = next[k]) {
            if (theshash[k] == pp) {                                 Yes. Return previous value.
                return k;
            }
            kprev = k;
        }
        k = ng ? garbg[--ng] : nn++ ;                               No. Get new integer.
        next[kprev] = k;
    }
    if (k >= nmax) throw("storing too many values");
    thehash[k] = pp;                                               Store the key at the new or previous integer.
    next[k] = -1;
    return k;
}
```

A word here about garbage collection. When a key is erased (by the routine immediately below), we want to make its integer available to future “sets,” so that `nmax` keys can always be stored. This is very easy to implement if we allocate a garbage array (`garbg`) and use it as a last-in first-out stack of available integers. The set routine above always checks this stack when it needs a new integer. (By the way, had we designed `Hashtable` with list elements linked by pointers, instead of

arrays, efficient garbage collection would have been more difficult to implement; see Stroustrup [1].)

```
hash.h  template<class keyT, class hfnT>
        Int Hashtable<keyT,hfnT>::ierase(const keyT &key) {
        Erase a key, returning the integer in 0..nmax-1 erased, or -1 if the key was not previously set.
            Int j,k,kprev;
            Ullong pp = hash.fn(&key);
            j = (Int)(pp % nhash);
            if (htable[j] == -1) return -1;      Key not previously set.
            kprev = -1;
            for (k = htable[j]; k != -1; k = next[k]) {
                if (theshash[k] == pp) {        Found key. Splice linked list around it.
                    if (kprev == -1) htable[j] = next[k];
                    else next[kprev] = next[k];
                    garbg[ng++] = k;            Add k to garbage stack as an available integer.
                    return k;
                }
                kprev = k;
            }
            return -1;                          Key not previously set.
        }
```

Finally, `Hashtable` has routines that reserve and relinquish integers in the range 0 to `nmax`. When an integer is reserved, it is guaranteed not to be used by the hash table. Below, we'll use this feature as a convenience in constructing a hash memory that can store more than one element under a single key.

```
hash.h  template<class keyT, class hfnT>
        Int Hashtable<keyT,hfnT>::ireserve() {
        Reserve an integer in 0..nmax-1 so that it will not be used by set(), and return its value.
            Int k = ng ? garbg[--ng] : nn++ ;
            if (k >= nmax) throw("reserving too many values");
            next[k] = -2;
            return k;
        }

        template<class keyT, class hfnT>
        Int Hashtable<keyT,hfnT>::irelinquish(Int k) {
        Return to the pool an index previously reserved by reserve(), and return it, or return -1 if it
        was not previously reserved.
            if (next[k] != -2) {return -1;}
            garbg[ng++] = k;
            return k;
        }
```

7.6.3 Hash Memory

The `Hashtable` class, above, implements Mr. Hacher's task. Building on it, we next implement *your* job in the dream, namely to do the actual storage and retrieval of arbitrary objects by arbitrary keys. This is termed a *hash memory*.

When you store into an ordinary computer memory, the value of anything previously stored there is overwritten. If you want your hash memory to behave the same way, then a hash memory class, `Hash`, derived from `Hashtable`, is almost trivial to write. The class is templated by three structure types: `keyT` for the key type; `e1T` for the type of the element that is stored in the hash memory; and `hfnT`, as before, for the object that encapsulates the hash function of your choice.

hash.h

```

template<class keyT, class elT, class hfnT>
struct Hash : Hashtable<keyT, hfnT> {
Extend the Hashtable class with storage for elements of type elT, and provide methods for
storing, retrieving, and erasing elements. key is passed by address in all methods.
    using Hashtable<keyT,hfnT>::iget;
    using Hashtable<keyT,hfnT>::iset;
    using Hashtable<keyT,hfnT>::ierase;
    vector<elT> els;

    Hash(Int nh, Int nm) : Hashtable<keyT, hfnT>(nh, nm), els(nm) {}
    Same constructor syntax as Hashtable.

    void set(const keyT &key, const elT &el)
    Store an element el.
        {els[iset(key)] = el;}

    Int get(const keyT &key, elT &el) {
    Retrieve an element into el. Returns 0 if no element is stored under key, or 1 for success.

        Int ll = iget(key);
        if (ll < 0) return 0;
        el = els[ll];
        return 1;
    }

    elT& operator[] (const keyT &key) {
    Store or retrieve an element using subscript notation for its key. Returns a reference that
    can be used as an l-value.
        Int ll = iget(key);
        if (ll < 0) {
            ll = iset(key);
            els[ll] = elT();
        }
        return els[ll];
    }

    Int count(const keyT &key) {
    Return the number of elements stored under key, that is, either 0 or 1.
        Int ll = iget(key);
        return (ll < 0 ? 0 : 1);
    }

    Int erase(const keyT &key) {
    Erase an element. Returns 1 for success, or 0 if no element is stored under key.
        return (ierase(key) < 0 ? 0 : 1);
    }
};

```

The operator[] method, above, is intended for two distinct uses. First, it implements an intuitive syntax for storing and retrieving elements, e.g.,

$$\text{myhash}[\text{some-key}] = \text{rhs}$$

for storing, and

$$\text{lhs} = \text{myhash}[\text{some-key}]$$

for retrieving. Note, however, that a small inefficiency is introduced, namely a superfluous call to get when an element is set for the first time. Second, the method returns a non-const reference that cannot only be used as an l-value, but also be pointed to, as in

